Appendix C: FSP Semantics

The semantics of basic *FSP* are defined in terms of Labeled Transition Systems (*LTSs*). In the body of the book, we have depicted the *LTS* that corresponds to an *FSP* process as a graph. In the following, we formally define what an *LTS* is and then describe the correspondence between *FSP* process expressions and *LTS*s. This correspondence is defined by the function:

$$lts : Exp \rightarrow \wp$$

where *Exp* is the set of *FSP* process expressions, and $\wp$ the set of *LTS*s. The function *lts* is defined inductively on the structure of *FSP* process expressions.

C.1 Labeled Transition System (LTS)

Let *States* be the universal set of states including π a designated *error* state, L be the universal set of labels, and $Act = L \cup \{\tau\}$, where τ is used to denote an internal action that cannot be observed by the environment of an *LTS*.

A finite *LTS* P is a quadruple $< S, A, \Delta, q >$ where:

- $S \subseteq States$ *is* a finite set of states.
- $A = \alpha P \cup \{\tau\}$, where $\alpha P \subseteq L$ denotes the *alphabet* of P.
- $\Delta \subseteq S - \{\pi\} \times A \times S$, denotes a transition relation that maps from a state and an action onto another state.
- $q \in S$ indicates the initial state of P.

The only *LTS* that is allowed to have the error state π as its initial state is $< \{\pi\}, Act, \{\}, \pi >$, named Π. The alphabet of this process $\alpha\Pi = L$.

An *LTS* $P = < S, A, \Delta, q >$ *transits* with action $a \in A$ into an *LTS* P', denoted as $P \xrightarrow{a} P'$, if:

- $P' = < S, A, \Delta, q' >$, where $q' \neq \pi$ and $(q, a, q') \in \Delta$, or
- $P' = \Pi$, and $(q, a, \pi) \in \Delta$.

We use $P \xrightarrow{a}$ to mean that $\exists P'$ such that $P \xrightarrow{a} P'$, and $P \not\xrightarrow{a}$ to mean that $\nexists P'$ such that $P \xrightarrow{a} P'$.

We define a set of designated end states $ES \subseteq States$ such that an *LTS* $P = < S, A, \Delta, q >$ is terminating if there is a state $e \in S$ and $e \in ES$ and $\nexists(e, a, q) \in \Delta$ for all $a \in A$.

C.2 Processes

In the following, *E* ranges over *FSP* process expressions, *Q* ranges over process identifiers, and *A*, *B* range over sets of observable actions (i.e. $A \subseteq L$ and $B \subseteq L$).

Process Definition:

Q = *E* means that *lts(Q)* $=_{def}$ *lts*(*E*).

Process Constants:

lts(END) = <{*e*}, {τ}, {}, *e*> where $e \in ES$
lts(STOP) = <{*s*}, {τ}, {}, *s*>
lts(ERROR) = Π

Prefix `->`**:**

If *lts*(*E*) = < *S, A,* Δ*, q* > and *E* is not `ERROR`

then *lts*(*a* **->** *E*) = < $S \cup \{p\}$, $A \cup \{a\}$, $\Delta \cup \{(p, a, q)\}$, *p* >, where $p \notin S$.
lts(*a* **-> ERROR**) = < {*p,* π}, {*a*}, {(*p, a,* π)}, *p* >, where $p \neq \pi$.

Choice `|` **:**

Let $1 \leq i \leq n$, and *lts*(E_i) = < S_i, A_i, Δ_i, q_i >,

then *lts*(a_1 -> E_1 | ... | a_n -> E_n)

= < $S \cup \{p\}$, $A \cup \{a_1 \ldots a_n\}$, $\Delta \cup \{(p, a_1, q_1) \ldots (p, a_n, q_n)\}$, *p* >,

where $p \notin S_i, S = \bigcup_i Si, A = \bigcup_i Ai, \Delta = \bigcup_i \Delta i$.
If E_i is ERROR then A_i = {}.

Alphabet Extension + :

If *lts*(*E*) =< *S, A,* Δ*, q* >,
then *lts*(*E* + *B*) =< *S,* $A \cup B$, Δ*, q* >.

Recursion:

We represent the *FSP* process defined by the recursive equation *X* = *E* as *rec*(*X* = *E*), where *X* is a variable in *E*. For

example, the process defined by the recursive definition X = (a –> X) is represented as *rec*(X =(a –> X)).

We use $E[X \leftarrow rec(X = E)]$ to denote the *FSP* expression that is obtained by substituting *rec*(X = E) for X in E. Then *lts*(*rec*(X = E)) is the smallest *LTS* that satisfies the following rule:

$$rec(X = (a \text{ -> } X)) \sim lts(E[X \leftarrow rec(X = E)])$$

where "~" is strong semantic equivalence defined in section C.6.1. Mutually recursive equations can be reduced to the simple form described above. For local processes, all occurrences of process variables are guarded by an action prefix and consequently, recursive definitions are guaranteed to have a fixed-point solution.

Sequential Composition:

If *lts*(P) = < S_p, A_p, Δ_p, q_p > is terminating with end state $e_p \in ES$
and *lts*(E) = < S_e, A_e, Δ_e, q_e >
then *lts*(P; E) = < $S_p \cup S_e$, $A_p \cup A_e$, $\Delta_p \cup \Delta_e$, q_p > where $q_e = e_p$.

C.3 Composite Processes

Before defining the meaning of composition in *FSP* and of the priority operators on composite processes, we must first describe the meaning of composition and priority in the underlying *LTS* model.

C.3.1 LTS Composition

The parallel composition $P||Q$ of two *LTS*s P and Q is defined as follows:

If $P = \Pi$ or $Q = \Pi$, then $P||Q = \Pi$.
For $P = < S_1, A_1, \Delta_1, q_1 >$ and $Q = < S_2, A_2, \Delta_2, q_2 >$, such that $P \neq \Pi$ and $Q \neq \Pi$,
$P||Q = < S_1 \times S_2, A_1 \cup A_2, \Delta, (q_1, q_2) >$,

where Δ is the smallest relation satisfying the rules:

Let $a \in Act$ in

$$\frac{P \xrightarrow{a} P'}{P||Q \xrightarrow{a} P'||Q} a \notin \alpha Q \qquad \frac{Q \xrightarrow{a} Q'}{P||Q \xrightarrow{a} P||Q'} a \notin \alpha P$$

$$\frac{P \xrightarrow{a} P', Q \xrightarrow{a} Q'}{P||Q \xrightarrow{a} P'||Q'} a \neq \tau$$

Parallel composition is both commutative and associative. Consequently, the order in which *LTS*s are composed is not significant.

In addition:
P || *lts*(END) = P, *lts*(END) || P = P and *lts*(END) || *lts*(END) = *lts*(END).

C.3.2 LTS Priority

The set of actions $B \subseteq Act$ are *high* priority in the *LTS* $P \ll B$, where $P = < S, A, \Delta', q >$.

$P \ll B = < S, A, \Delta, q >$ where Δ is the smallest relation satisfying the rule:

Let $a, b \in Act$ in

$$\frac{P \xrightarrow{a} P'}{P \ll B \xrightarrow{a} P' \ll B} \text{if}((a \in B) \text{ or } (\forall b \in B, P \not\xrightarrow{b}))$$

The set of actions $B \subseteq Act$ are *low* priority in the *LTS* $P \gg B$, where $P = < S, A, \Delta', q >$.

$P \gg B = < S, A, \Delta, q >$ where Δ is the smallest relation satisfying the rule:

Let $a, b \in Act$ in

$$\frac{P \xrightarrow{a} P'}{P \gg B \xrightarrow{a} P' \gg B} \text{ if}((a \notin B) \text{ or } (\forall b \in (A - B), P \not\xrightarrow{b}))$$

C.3.3 FSP Composition and Priority

Using the definitions for *LTS* composition and priority, we can now simply define the meaning of composition and priority in *FSP*. In the following, *CE* refers to *FSP* composition expressions of the form (Q_1 ||…||Q_n) and *Q* refers to the identifier of a process or composite process.

Parallel Composition || :

lts(Q_1 || Q_2) = *lts*(Q_1) || *lts*(Q_2).

Priority High << :

lts(*CE*<<*B*) = *lts*(*CE*) << *B*.

Priority Low >> :

lts(*CE* >>*B*) = *lts*(*CE*) >> *B*.

C.4 Common Operators

To define the *FSP* re-labeling, hiding and interface operators, which can be applied to processes and composite processes, we first describe the meaning of re-labeling and hiding in the underlying *LTS* model.

C.4.1 Re-Labeling

Re-labeling applies a relation over action labels $R \subseteq L \times L$, to an *LTS* $P = < S, A, \Delta, q >$ such that:

$$P/R = < S, (A - B_1) \cup B_2, (\Delta - \Delta_1) \cup \Delta_2, q > .$$

where

$B_1 = \{a \in A | \exists a'.(a, a') \in R\}$,

$B_2 = \{a' | \exists a \in A.(a, a') \in R\}$,

$\Delta_1 = \{(p, a, p') \in \Delta | a \in B_1\}$, and

$\Delta_2 = \{(p, a', p') | (p, a, p') \in \Delta_1 \wedge (a, a') \in R\}$.

C.4.2 Hiding

The set of actions $B \subseteq Act$ are *hidden* in the *LTS* $P\backslash B$, where $P = < S, A, \Delta, q >$.

If $P = \Pi$ then $P\backslash B = \Pi$ otherwise

$P\backslash B = < S, (A–B) \cup \{\tau\}, \Delta, q >$

where Δ is the smallest relation satisfying the rule:

Let $a \in Act$ in

$$\frac{P \xrightarrow{a} P'}{P\backslash B \xrightarrow{\tau} P'\backslash B}\, a \in B \qquad \frac{P \xrightarrow{a} P'}{P\backslash B \xrightarrow{a} P'\, B}\, a \notin B$$

C.4.3 FSP Re-Labeling, Hiding and Interface

Re-labeling /:

lts(EI R) = lts(E)/R.
lts((*Q*$_1$ || ... || *Q*$_n$)*I R) = lts*(*Q*$_1$)*/R* || ... || *lts*(*Q*$_n$)*/R*

Hiding\:

lts(E\B) = lts(E)\B.

Interface @:

lts(E@ I) = lts(E)\B where *B* = α*(lts(E)) – I*.

C.5 Safety Properties

A safety property *Q* in *FSP* is represented by an *image* of the *LTS* of the process expression that defines the property. The image *LTS* has each state of the original *LTS* and has a transition from each state for every action in the alphabet of the original. Transitions added to the image *LTS* are to the error state.

property *Q* = *E*:

lts(Q) $=_{\text{def}}$ *image(lts(E))*,

for an *LTS P* =< *S, A,* Δ*, p* >, *image(P)* = < *S* ∪ {*π*}, *A*, Δ′, *q* >, where Δ′ = Δ ∪ {*(s, a, π)*|*s* ∈ *S*, *a* ∈ *A*, and ∄*s′* ∈ *S*: *(s, a, s′)* ∈ Δ}.

C.6 Semantic Equivalences

Minimization of the *LTS* corresponding to an *FSP* process definition results in a semantically equivalent *LTS*. The equivalence relations used in performing minimization are defined in the following sections.

C.6.1 Strong Equivalence

Strong semantic equivalence equates *LTS*s that have identical behavior when the occurrence of all their actions can be observed, including that of the silent action τ.

Let $\wp$ be the universal set of *LTS*s. *Strong semantic equivalence* "**~**" is the union of all relations $R \subseteq \wp \times \wp$ satisfying that *(P, Q)* $\in$ ***R*** implies:

1. $\alpha P = \alpha Q$;
2. $\forall a \in Act$:
   - $P \xrightarrow{a} P'$ implies $\exists Q', Q \xrightarrow{a} Q'$ and *(P′, Q′)* $\in$ ***R***.
   - $Q \xrightarrow{a} Q'$ implies $\exists$ *P′*, $P \xrightarrow{a} P'$ and *(P′, Q′)* $\in$ ***R***.
3. *P* = Π iff *Q* = Π.

The *LTSA* tool performs minimization using strong equivalence if an *LTS* contains no silent actions (τ). For an *LTS P*, without τ-actions:

minimized(P) ~ P.

C.6.2 Weak Equivalence

Weak semantic equivalence equates systems that exhibit the same behavior to an external observer who cannot detect the occurrence of τ-actions.

Let $P \stackrel{a}{\Rightarrow} P'$ denote $P \xrightarrow{\tau^* a \tau^*} P'$, where τ^* means a sequence of zero or more τs. Then *weak (or observational) semantic equivalence* "$\approx$" is

the union of all relations $R \subseteq \wp \times \wp$ satisfying that *(P, Q)* $\in$ ***R*** implies:

1. $\alpha P = \alpha Q$;
2. $\forall\ a \in L \cup \{\epsilon\}$, where $L = Act - \{\tau\}$, and ϵ is the empty sequence:
   - $P \stackrel{a}{\Rightarrow} P'$ implies $\exists Q'$, $Q \stackrel{a}{\Rightarrow} Q'$ and *(P′, Q′)* $\in$ ***R***.
   - $Q \stackrel{a}{\Rightarrow} Q'$ implies $\exists P'$, $P \stackrel{a}{\Rightarrow} P'$ and *(P′, Q′)* $\in$ ***R***.
3. $P = \Pi$ iff $Q = \Pi$.

The *LTSA* tool performs minimization using weak equivalence if an *LTS* contains silent actions (τ). For an *LTS P*, with τ-actions:

$$minimized(P) \approx P.$$

Both strong and weak equivalence are congruences with respect to the composition, re-labeling, and hiding operators. This means that strongly or weakly equivalent components may substitute one another in any system constructed with these operators, without affecting the behavior of the system with respect to strong or weak equivalence, respectively.

C.7 Fluent Linear Temporal Logic (FLTL)

C.7.1 Linear Temporal Logic

Given a set of atomic propositions $\wp$, a well-formed LTL formula is defined inductively using the standard Boolean operators **!** (not), **||** (or), **&&** (and), and the temporal operators **X** (next) and **U** (strong until) as follows:

- each member of $\wp$ is a formula,
- if ϕ and ψ are formulae, then so are **!** ϕ, ϕ **||** ψ, ϕ **&&** ψ, **X** ϕ, ϕ **U** ψ.

An interpretation for an LTL formula is an infinite word $w = x_0x_1x_2 \ldots$ over $2^{\wp}$. In other words, an interpretation maps to each instant of time a set of propositions that hold at that instant. We write w_i for the suffix of w starting at x_i. LTL semantics is then defined inductively as follows:

- $w \models p$ iff $p \in x_0$, for $p \in \wp$
- $w \models$ **!** ϕ iff not $w \models \phi$
- $w \models \phi$ **||** ψ iff ($w \models \phi$) or ($w \models \psi$)
- $w \models \phi$ **&&** ψ iff ($w \models \phi$) and ($w \models \psi$)
- $w \models$ **X** ϕ iff $w_1 \models \phi$
- $w \models \phi$ **U** ψ iff $\exists i \geq 0$, such that: $w_i \models \psi$ and $\forall\ 0 \leq j < i$, $w_j \models \phi$

We introduce the abbreviations “true $\equiv \phi$ || **!** ϕ” and “false $\equiv$ **!** true”. Implication −> and equivalence <−> are defined as:

$$\varphi \text{ -> } \psi \equiv \text{ ! } \varphi \text{ || } \psi,$$

$$\text{and} \quad \varphi \text{ <-> } \psi \equiv (\varphi \text{ -> } \psi) \text{ && } (\varphi \text{ <- } \psi)$$

Temporal operators <> (eventually), `[]` (always), and `W` (weak until) are defined in terms of the main temporal operators:

$$\text{<>}\varphi \equiv \text{ true U}$$

$$\text{[]}\varphi \equiv \text{ !<>! } \varphi,$$

$$\text{and} \quad \varphi \text{ W } \psi \equiv ((\varphi \text{ U } \psi) \text{ || [] } \varphi).$$

C.7.2 Fluents

We define a fluent *Fl* by a pair of sets, a set of initiating actions I_{Fl} and a set of terminating actions T_{Fl}:

$$Fl \equiv \langle I_{Fl}, T_{Fl} \rangle \text{ where } I_{Fl}, T_{Fl} \subset Act \text{ and } I_{Fl} \cap T_{Fl} = \emptyset$$

In addition, a fluent *Fl* may initially be true or false at time zero as denoted by the attribute $Initially_{Fl}$.

The set of atomic propositions from which FLTL formulae are built is the set of fluents Φ. Therefore, an interpretation in FLTL is an infinite word over 2^{Φ}, which assigns to each time instant the set of fluents that hold at that time instant. An infinite word $< a_0 a_1 a_2 \cdots >$ over *Act* (i.e. an infinite trace of actions) also defines an FLTL interpretation $< f_0 f_1 f_2 \cdots >$ over 2^{Φ} as follows:

$\forall i \in N, \forall\ Fl \in \Phi, Fl \in f_i$ iff either of the following holds

– $Initially_{Fl} \wedge (\forall k \in N \cdot 0 \leq k \leq i, a_k \notin T_{Fl})$

– $\exists j \in N : ((j \leq i) \wedge (a_j \in I_{Fl}) \wedge (\forall k \in N \cdot j < k \leq i, a_k \notin T_{Fl}))$

In other words, a fluent holds at a time instant if and only if it holds initially or some initiating action has occurred, and, in both cases, no terminating action has yet occurred. Note that the interval over which

a fluent holds is *closed* on the left and *open* on the right, since actions have immediate effect on the values of fluents. Since the sets of initiating and terminating actions are disjoint, the value of a fluent is always deterministic with respect to a system execution.

Every action *a* implicitly defines a fluent whose initial set of actions is the singleton {*a*} and whose terminating set contains all other actions in the alphabet of a process $A \subseteq Act$:

$$Fluent(a) \equiv \langle \{a\}, A - \{a\} \rangle \; Initially_a = \text{false}$$

Fluent(*a*) becomes true the instant *a* occurs and becomes false with the first occurrence of a different action. It is often more succinct in defining properties to declare a fluent implicitly for a set of actions as in:

$$Fluent(S) \equiv \langle S, A - S \rangle \; Initially_S = \text{false}, \text{ where } S = \{a_0, a_1, \ldots a_n\}$$

This is equivalent to a_0 || a_1 || ... || a_n where a_i represents the implicitly defined *Fluent*(a_i).